

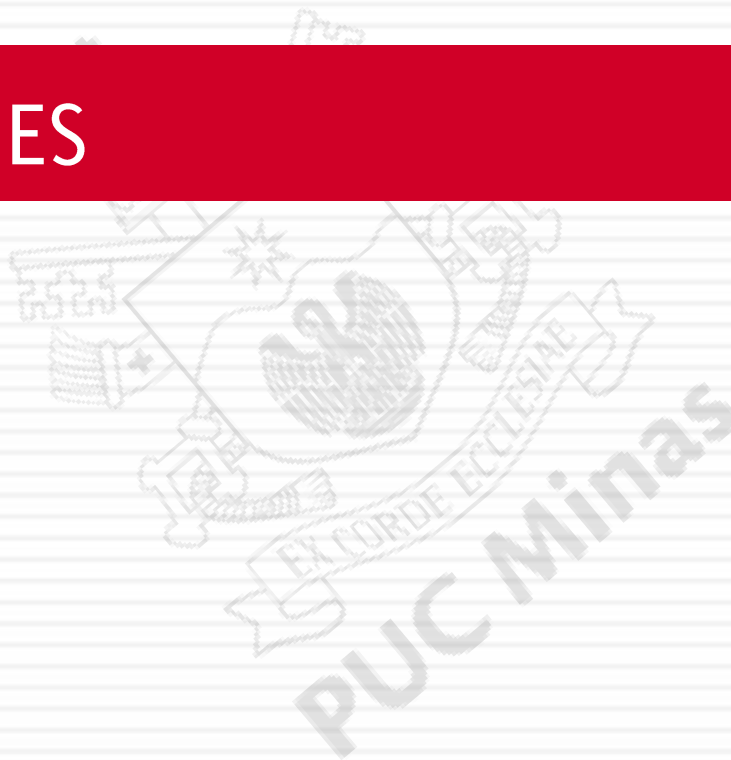
PROGRAMAÇÃO MODULAR

COLEÇÕES, MAPEAMENTOS E FLUXOS DE DADOS

PROF. JOÃO CARAM

2

COLEÇÕES



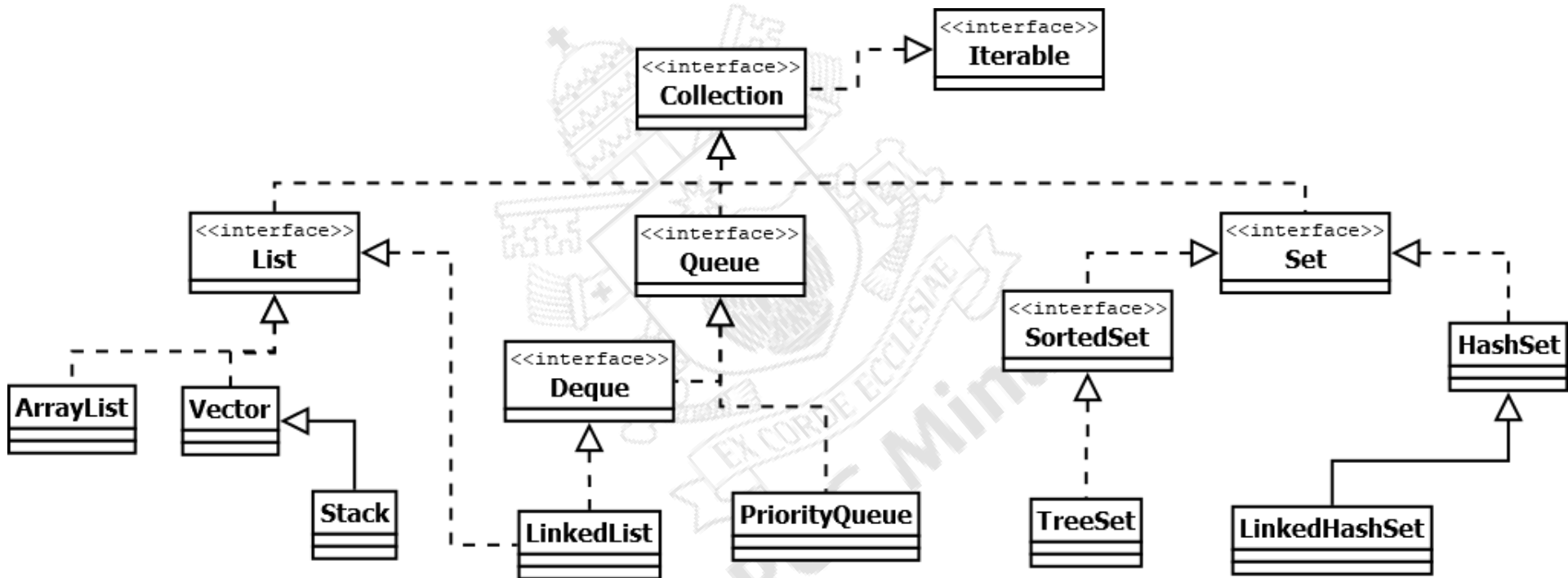
COLEÇÕES

3

- ▣ Objetos que representam um grupo de objetos tratados como uma única entidade.
 - ▣ “*Estruturas de dados*”.
- ▣ *Java collection framework* (JCF):
 - ▣ Arquitetura de componentes para permitir manipulação de coleções de maneira independente e abstrata.

JCF – COLEÇÕES

4



COLEÇÕES NO JCF

5

- Implementam a interface “Iterable”;
- Processamento típico com *forEach*:

```
for(Object obj: collection)  
    facaAlgumaCoisa(obj);
```

6

UTILIZANDO COLEÇÕES / JCF



INTERFACE *COLLECTION*<E>

boolean add(E elemento) -- **opcional**

boolean remove(E elemento) -- **opcional**

void clear() -- **opcional**

int size()

boolean isEmpty()

boolean contains(Object elemento)

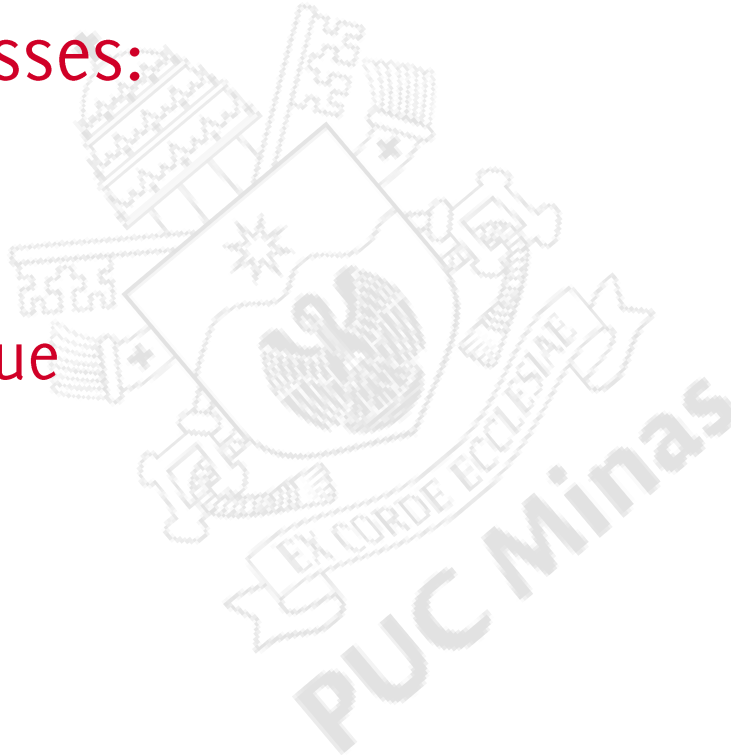
Object[] toArray()

COLEÇÕES

8

■ Algumas classes:

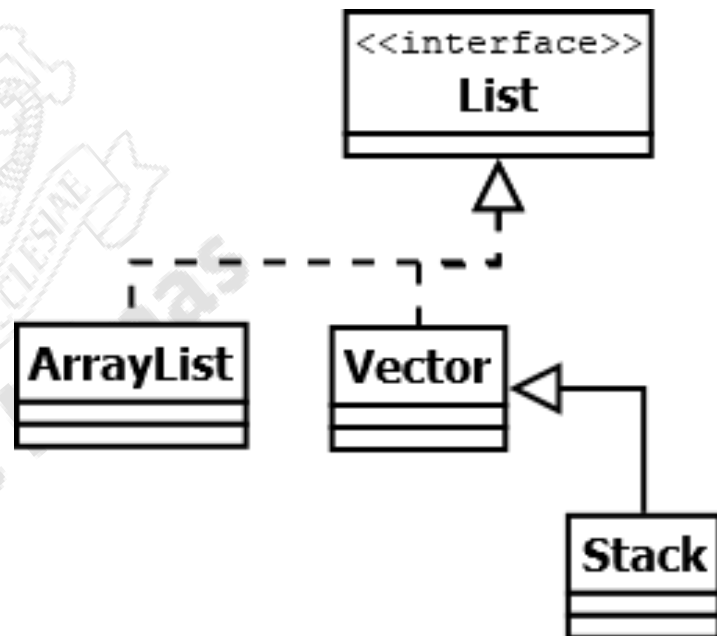
- ArrayList
- LinkedList
- PriorityQueue
- HashSet
- SortedSet



CLASSE *ARRAYLIST*<E>

9

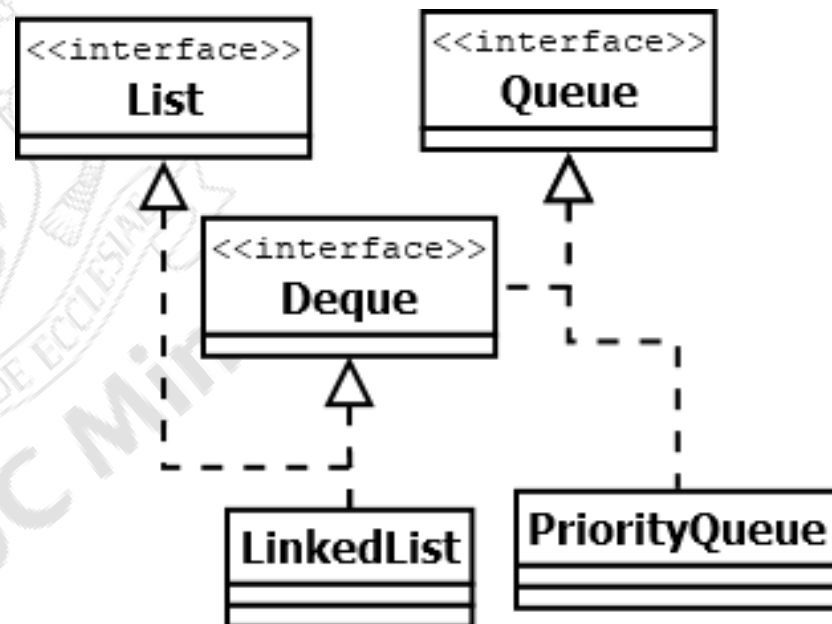
- Implementação de uma lista usando um *array*;
 - *Acesso por índice.*
- Tamanho pré-definido;
- Auto-redimensionável;
 - Cuidado com desempenho.



CLASSE *LINKEDLIST*<E>

10

- Implementa de lista e/ou fila dinâmicas:
- Tamanho flexível.
- Métodos para acesso nas extremidades.



A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto 'EX CORDE ECCLESIAE', and the text 'PUC Minas' below it.

{ Quero ver código!! }

ALGORITMOS ASSOCIADOS A COLEÇÕES

12

▣ Collections:

- ▣ classe com implementação *static* de algoritmos comuns aplicáveis a objetos que se enquadrem como coleções.

sort

shuffle

reverse

fill

copy

swap

binarySearch

frequency

disjoint

min

max

The background of the slide features a large, faint watermark of the PUC Minas logo. The logo is a crest with a star, a banner that reads "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

14

MAPEAMENTOS



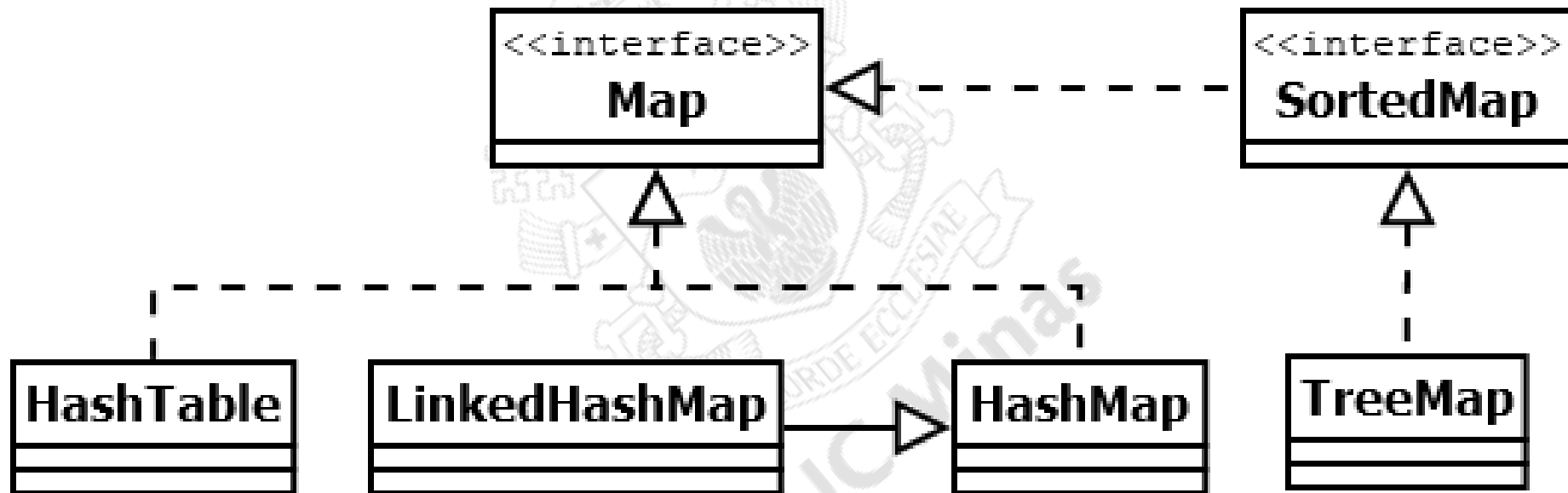
MAPAS E MAPEAMENTOS

15

- Não implementam a interface *Collection*, mas oferecem operações semelhantes.
 - Não são iteráveis;
- Associação de chaves e valores/objetos:
 - Parâmetro duplo.
- Localização rápida de elementos.

JCF - MAPEAMENTOS

16



17

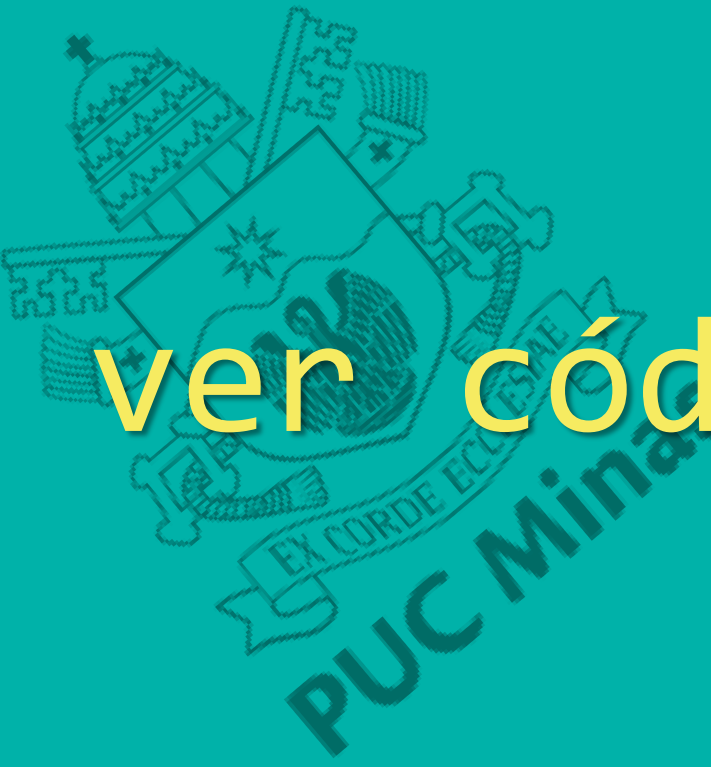
UTILIZANDO MAPAS / JCF



CLASSE *HASHMAP*<K,V>

18

- Mapeamento rápido – mapa / tabela *hash*.
 - Busca mais rápida.
 - Não garante ordem de entrada e nem ordem constante!



{ Quero ver código!! }

20

FLUXOS DE DADOS (*STREAMS*)



STREAMS

21

- ▣ Sequência de elementos na qual é possível realizar operações de agregação sequenciais ou paralelas.
- ▣ Transporta os valores via um *pipeline*.
 - ▣ *Stream*: fluxo de dados.
- ▣ Um *stream* pode ser gerado de uma coleção, mas não é uma coleção.

STREAMS X COLEÇÕES

22

- **Coleção:**
 - Gerência e acesso eficientes a dados e seus elementos.
- **Streams:**
 - Descrição de operações computacionais, em geral agregadoras, sobre os elementos de uma fonte.

STREAMS

23

- ▣ Arquitetura do *stream*:
 - ▣ Fonte de dados: coleção, E/S, função geradora...
 - ▣ Operações intermediárias;
 - ▣ Operação final (terminal).

STREAMS – EXEMPLO

```
Collection<Carro> carros = new .... ;
```

```
int sum = carros.stream()  
    .filter(c -> c.kmRodados() > 100_000)  
    .mapToInt(c -> c.kmRodados())  
    .sum();
```


STREAMS – EXEMPLO

```
Collection<Carro> carros = new .... ;
```

```
int sum = carros.stream()  
    .filter(c -> c.kmRodados() > 100_000)  
    .mapToInt(c -> c.kmRodados())  
    .sum();
```

Cria um stream com
esta fonte de dados

STREAMS – EXEMPLO

```
Collection<Carro> carros = new .... ;
```

```
int sum = carros.stream()  
    .filter(c -> c.kmRodados() > 100_000)  
    .mapToInt(c -> c.kmRodados())  
    .sum();
```

Operação intermediária:
Filtro: todos os carros
com mais de 100.000 km

STREAMS – EXEMPLO

```
Collection<Carro> carros = new .... ;
```

```
int sum = carros.stream()  
    .filter(c -> c.kmRodados() > 100_000)  
    .mapToInt(c -> c.kmRodados())  
    .sum();
```

Operação intermediária:
Mapeamento: de objeto
(carro) para inteiro (km)

STREAMS – EXEMPLO

```
Collection<Carro> carros = new .... ;
```

```
int sum = carros.stream()  
    .filter(c -> c.kmRodados() > 100_000)  
    .mapToInt(c -> c.kmRodados())  
    .sum();
```

Operação terminal:
Soma os inteiros
recebidos via pipeline

STREAMS VS ITERATIVO

```
int kmTotais = carros.stream()
    .filter(c -> c.kmRodados() > 100_000)
    .mapToInt(c -> c.kmRodados())
    .sum();
```

```
int kmTotais = 0;
for(int i=0; i< carros.length; i++){
    if(carros[i].kmRodados() > 100_000)
        kmTotais += carros[i].kmRodados();
}
return kmTotais;
```

STREAMS VS ITERATIVO

```
int kmTotais = carros.stream()
    .filter(c -> c.kmRodados() > 100_000)
    .mapToInt(c -> c.kmRodados())
    .sum();
```

```
int kmTotais = 0;
for(int i=0; i< carros.length; i++){
    if(carros[i].kmRodados() > 100_000)
        kmTotais += carros[i].kmRodados();
}
return kmTotais;
```

O que vocês têm a me dizer sobre estes dois códigos?

{ Quero ver código!! }



STREAMS

32

- Por que *streams*?
 - “Métodos guiados por dados”.
 - *Pipeline* de dados iniciado quando última operação é iniciada.
 - *Streams* paralelas.
 - Redução da complexidade de escrita de código paralelo.
 - Legibilidade.
 - Declarativo x imperativo.

33

EM RESUMO...



STREAM: OPERAÇÕES INTERMEDIÁRIAS

34

distinct – outro stream com os elementos distintos

filter – outro stream com filtro/predicado aplicado

limit – retorna um stream com máximo de elementos

map – mapeamentos diversos

skip – outro stream após ‘saltar’ N elementos

sorted – outro stream com a ordenação deste

STREAM: OPERAÇÕES TERMINAIS

35

allMatch, **anyMatch** – booleano para verificar predicado

average – média, para valores numéricos

count – long com o número de elementos

findFirst – primeiro elemento do stream

forEach – ação aplicada a cada elemento do stream

STREAM: OPERAÇÕES TERMINAIS

36

`min`, `max` – mínimo e máximo por um comparador

`reduce` – reduz os elementos por associação acumulativa

`sum` – soma, para valores numéricos

`summaryStatistics` – vários dados sumarizados

`toArray`, `toList`, `collect` – conversões

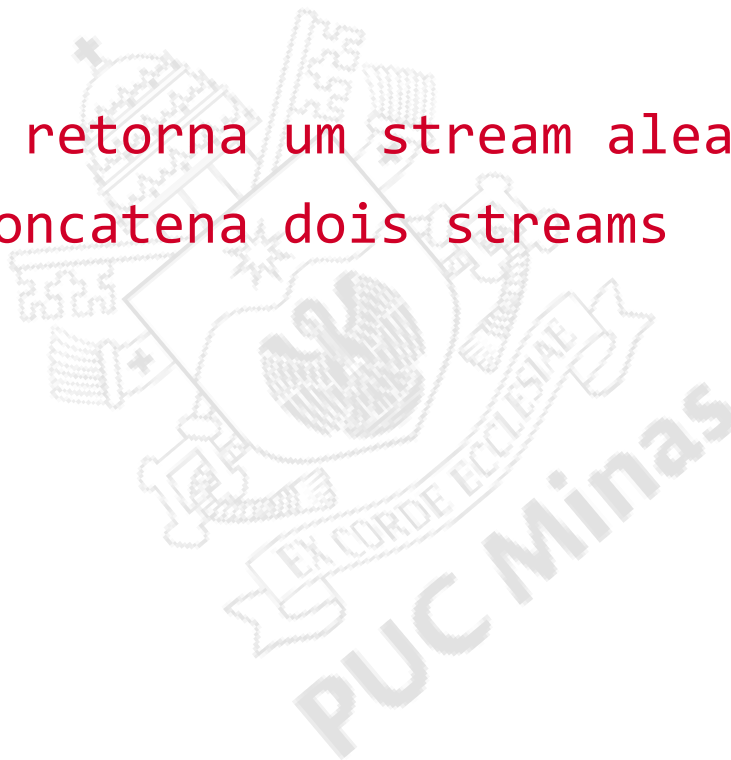
STREAM: MÉTODOS STATIC

37

static:

generate – retorna um stream aleatório infinito

concat – concatena dois streams



STREAMS - EXEMPLO

```
double areaMedia =  
    figs.stream().filter(p -> p.toString().contains("Quadrado"))  
        .mapToDouble(Figura::area)  
        .average()  
        .getAsDouble();
```

1. Estabelece a fonte de dados a partir da coleção

STREAMS - EXEMPLO

```
double areaMedia =  
    figs.stream().filter(p -> p.toString().contains("Quadrado"))  
        .mapToDouble(Figura::area)  
        .average()  
        .getAsDouble();
```

1. Estabelece a fonte de dados a partir da coleção
2. Filtra mantendo só objetos descritos como *Quadrado*

STREAMS - EXEMPLO

```
double areaMedia =  
    figs.stream().filter(p -> p.toString().contains("Quadrado"))  
        .mapToDouble(Figura::area)  
        .average()  
        .getAsDouble();
```

1. Estabelece a fonte de dados a partir da coleção
2. Filtra mantendo só objetos descritos como *Quadrado*
3. Mapeia o resultado do filtro para *double* pela área (objeto -> valor)

STREAMS - EXEMPLO

```
double areaMedia =  
    figs.stream().filter(p -> p.toString().contains("Quadrado"))  
        .mapToDouble(Figura::area)  
        .average()  
        .getAsDouble();
```

1. Estabelece a fonte de dados a partir da coleção
2. Filtra mantendo só objetos descritos como *Quadrado*
3. Mapeia o resultado do filtro para *double* pela área (objeto -> valor)
4. Faz a média do valor mapeado para um *OptionalDouble*

STREAMS - EXEMPLO

```
double areaMedia =  
    figs.stream().filter(p -> p.toString().contains("Quadrado"))  
        .mapToDouble(Figura::area)  
        .average()  
        .getAsDouble();
```

1. Estabelece a fonte de dados a partir da coleção
2. Filtra mantendo só objetos descritos como *Quadrado*
3. Mapeia o resultado do filtro para *double* pela área (objeto -> valor)
4. Faz a média do valor mapeado para um *OptionalDouble*
5. Retorna a média como *double*

OBRIGADO.

DÚVIDAS?

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE